

Data “Slicing and Dicing” বলতে কী বোঝায়?

এখানে মূল কথা হলো:

Dataset-এর ডেটা কেটে-ছেঁটে, ফিল্টার করে, নাম বদলে, পরিষ্কার করে, ছোট **sample** নিয়ে—**analysis**-এর জন্য প্রস্তুত করা।

Datasets লাইব্রেরিতে **Pandas**-এর মতোই অনেক ফাংশন আছে, যেগুলো দিয়ে ডেটা ম্যানিপুলেট করা যায়।

এই উদাহরণে কোন **dataset** ব্যবহার করা হয়েছে?

এখানে **Drug Review Dataset** ব্যবহার করা হয়েছে।

এই dataset-এ আছে:

- রোগীরা কোন drug ব্যবহার করেছে
- কোন condition-এর জন্য করেছে
- সেই drug সম্পর্কে review
- রোগী কত rating দিয়েছে (১০-এর মধ্যে)
- আর কতজন review-টাকে useful মনে করেছে

অর্থাৎ এটা একধরনের **patient drug review dataset**।

Step 1: ডেটা **download** এবং **extract** করা

```
!wget "https://archive.ics.uci.edu/ml/machine-learning-databases/00462/drugsCom_raw.zip"
!unzip drugsCom_raw.zip
```

এখানে:

- **wget** দিয়ে zip file নামানো হচ্ছে
- **unzip** দিয়ে zip file extract করা হচ্ছে

Step 2: TSV file load করা

```
from datasets import load_dataset
```

```
data_files = {"train": "drugsComTrain_raw.tsv", "test": "drugsComTest_raw.tsv"}
```

```
drug_dataset = load_dataset("csv", data_files=data_files, delimiter="\t")
```

এখানে একটা গুরুত্বপূর্ণ বিষয় আছে:

- **TSV** মানে **Tab Separated Values**
- **CSV**-র মতোই, কিন্তু comma-এর বদলে **tab** separator ব্যবহার করে

তাই `load_dataset("csv", ...)` ব্যবহার করা গেলেও, extra করে বলতে হয়েছে:

```
delimiter="\t"
```

মানে কলামগুলো tab দিয়ে আলাদা করা আছে।

Step 3: ছোট random sample নেওয়া

```
drug_sample = drug_dataset["train"].shuffle(seed=42).select(range(1000))  
drug_sample[:3]
```

এখানে কী হচ্ছে?

`shuffle(seed=42)`

ডেটা এলোমেলোভাবে মিশিয়ে দিচ্ছে।

`select(range(1000))`

তারপর shuffled dataset থেকে প্রথম 1000টা row নিচ্ছে।

কেন এটা করা হয়?

কারণ বড় dataset নিয়ে কাজ শুরু করার আগে ছোট sample দেখে বোঝা যায়:

- ডেটার ধরন কী
- কোথায় সমস্যা আছে
- cleaning লাগবে কি না
- কোন columns important

এটা data analysis-এর খুব common good practice।

seed=42 কেন ব্যবহার করা হয়েছে?

`shuffle()` করলে ডেটা randomভাবে বদলে যায়।

কিন্তু `seed=42` দিলে একই code আবার চালালে একইভাবে **shuffle** হবে।

এটাকে বলে **reproducibility**।

মানে:

- আজ run করলে যা পেলো
- কাল আবার run করলেও একই sample পাবে

sample দেখে কী কী সমস্যা ধরা পড়ল?

উদাহরণ দেখে ৩টা সমস্যা চোখে পড়ে:

1. Unnamed: 0 column

এটা দেখতে এমন লাগছে যেন patient-এর একটা anonymous ID

অর্থাৎ আসল নাম নয়, বরং প্রতিটি রোগীর আলাদা ID।

2. condition column

এখানে label-গুলো mixed case:

- কিছু uppercase
- কিছু lowercase
- কিছু ঠিকমতো formatted না

যেমন:

- ADHD
- ibromyalgia
- Inflammatory Conditions

এতে একই condition একাধিকভাবে লেখা থাকতে পারে।

তাই normalize করা দরকার।

3. review column

Review-গুলোতে সমস্যা আছে:

- লম্বা-ছোট mixed length
- `\r\n` আছে → line break
- HTML character codes আছে, যেমন `'`

এগুলো text cleaning-এর সময় ঠিক করতে হবে।

Step 4: **Unnamed: 0** আসলেই unique ID কি না যাচাই করা

```
for split in drug_dataset.keys():  
    assert len(drug_dataset[split]) == len(drug_dataset[split].unique("Unnamed: 0"))
```

এখানে কী হচ্ছে?

`len(drug_dataset[split])`

মোট row সংখ্যা

`unique("Unnamed: 0")`

এই column-এ কয়টা unique value আছে

যদি row সংখ্যা = unique ID সংখ্যা হয়,
তাহলে বোঝা যায় প্রতিটি row-এর জন্য আলাদা ID আছে।

অর্থাৎ **Unnamed: 0** সম্ভবত patient ID।

Step 5: column-এর নাম বদলানো

```
drug_dataset = drug_dataset.rename_column(  
    original_column_name="Unnamed: 0", new_column_name="patient_id"  
)
```

এখানে `Unnamed: 0` নামটা খুব meaningful না।

তাই এটাকে `rename` করে `patient_id` করা হয়েছে।

লাভ কী?

Code পড়তে সহজ হয়।

Column-এর অর্থ পরিষ্কার হয়।

আগে:

- `Unnamed: 0`

পরে:

- `patient_id`

এটা data cleaning-এর খুব গুরুত্বপূর্ণ অংশ।

Step 6: `condition` column clean করতে গিয়ে error

```
def lowercase_condition(example):  
    return {"condition": example["condition"].lower()}
```

```
drug_dataset.map(lowercase_condition)
```

এখানে লক্ষ্য:

সব condition-কে lowercase করা।

যেমন:

- `ADHD` → `adhd`
- `Inflammatory Conditions` → `inflammatory conditions`

কিন্তু error এসেছে:

AttributeError: 'NoneType' object has no attribute 'lower'

এই **error**-এর মানে কী?

এর মানে কিছু row-তে `condition` এর value `None` আছে।

`None` হলো empty / missing value।

এখন `.lower()` শুধু string-এর ওপর কাজ করে।

`None.lower()` করলে error হবেই।

Step 7: `None` value-ওয়ালা row বাদ দেওয়া

```
drug_dataset = drug_dataset.filter(lambda x: x["condition"] is not None)
```

এখানে `filter()` ব্যবহার করা হয়েছে।

এর কাজ:

যেসব row condition `None` না, শুধু সেগুলো রাখবে।

অর্থাৎ:

- condition আছে → রাখা হবে
- condition নেই (`None`) → বাদ যাবে

lambda function কী?

`lambda` হলো Python-এ ছোট, নামবিহীন function।

General form:

lambda arguments: expression

উদাহরণ:

```
lambda x: x * x
```

এটা এমন একটি function যা কোনো সংখ্যার square বের করে।

যদি 3 দাও:

```
(lambda x: x * x)(3)
```

আউটপুট হবে:

9

আরেকটা উদাহরণ:

```
(lambda base, height: 0.5 * base * height)(4, 8)
```

এটা triangle-এর area বের করছে।

ফলাফল:

16.0

এখানে **lambda** কেন ব্যবহার করা হয়েছে?

কারণ function-টা খুব ছোট:

```
lambda x: x["condition"] is not None
```

এটার মানে:

- প্রতিটি row (**x**) দেখো
- যদি **condition** ফাঁকা না হয়, তাহলে **True**
- না হলে **False**

filter() সেই row-গুলো রাখে যেগুলো **True** দেয়।

Step 8: এখন **condition lowercase** করা

```
drug_dataset = drug_dataset.map(lowercase_condition)
drug_dataset["train"]["condition"][:3]
```

এখন যেহেতু **None** row বাদ গেছে, **.lower()** safely কাজ করবে।

ফলাফল:

```
['left ventricular dysfunction', 'adhd', 'birth control']
```

অর্থাৎ condition label-গুলো এখন normalized।

এখানে **map()** কী করে?

map() প্রতিটি row-এর ওপর একটি function apply করে।

যেমন এখানে:

```
def lowercase_condition(example):  
    return {"condition": example["condition"].lower()}
```

এর মানে প্রতিটি example-এর **condition** field lowercase করে নতুন value সেট করা।

সহজ ভাষায়:

map() = dataset-এর প্রতিটি row-র ওপর একই কাজ চালানো

filter() আর **map()** এর পার্থক্য

filter()

row রাখবে না বাদ দেবে সেটা ঠিক করে

উদাহরণ:

```
drug_dataset.filter(lambda x: x["condition"] is not None)
```

map()

row-এর data পরিবর্তন করে

উদাহরণ:

```
drug_dataset.map(lowercase_condition)
```